

Música

Academia de Música SonoRitmo

1. Contexto del caso

La **Academia de Música SonoRitmo** requiere un sistema para administrar las clases que dicta y registrar el costo de cada una. La academia trabaja con distintos tipos de clases; aunque todas comparten información básica como código de clase, duración en meses y cupo máximo, el costo varía según el tipo de clase.

Adicionalmente, ciertas clases son candidatas a contar con una sala de ensayo privada, lo que implica gestionar su disponibilidad de forma diferenciada. El sistema debe modelar esta situación aplicando los principios de la Programación Orientada a Objetos, de manera que el comportamiento compartido y el comportamiento específico de cada tipo de clase queden correctamente organizados en la jerarquía de clases.

2. Información necesaria

Cada clase debe incluir constructor(es), métodos accedadores (getters), mutadores (setters) y el método `toString()`, el cual debe incluir únicamente el código de clase y la duración en meses. Los constructores deben delegar la asignación de valores a los métodos `setter`, de modo que las validaciones definidas se apliquen desde la construcción del objeto.

- Todas las clases del sistema comparten código de clase, duración en meses y cupo máximo. Dado que el costo se determina de manera diferente para cada tipo de clase, el sistema debe contemplar un mecanismo que permita calcular ese costo de forma general sin conocer el tipo específico del objeto en cada momento.
- Las **clases individuales** agregan el instrumento principal, si el estudiante rindió una evaluación de nivel y un indicador de sala de ensayo privada asignada. Al participar en el servicio de sala privada de la academia, deben implementar el contrato definido por la interfaz **ConSalaPrivada**.
- Las **clases grupales** agregan un atributo que indica la cantidad de integrantes, condición que incide en el costo de la clase.
- El gestor de la academia es la clase responsable de administrar la colección de clases registradas y exponer las operaciones sobre ella.

3. Métodos especializados

El sistema debe ser capaz de calcular el costo para cualquier clase registrada, independientemente del tipo. Cada tipo de clase determina ese costo según su propia lógica:

- **Clase Individual:** el costo base es \$60.000. Si el estudiante no rindió una evaluación de nivel, ese costo se incrementa en un 15%.
- **Clase Grupal:** el costo base es \$25.000. Si la cantidad de integrantes supera 6, el costo se incrementa en un 10%.

Interfaz ConSalaPrivada

Define el contrato que deben cumplir las clases que participan en el servicio de sala de ensayo privada de la academia. Está compuesta por dos operaciones:

- La primera permite consultar en cualquier momento si la clase tiene una sala privada asignada.
- La segunda permite asignar una sala privada a la clase.

Esta interfaz está diseñada de manera que, en el futuro, cualquier tipo de clase pueda incorporarse al servicio de sala privada sin necesidad de modificar la jerarquía de clases existente. Por ahora, solo las clases individuales implementan este contrato.

4. Validaciones esperadas

Las validaciones deben aplicarse en los métodos setter de cada clase. Cuando una condición no se cumpla, debe lanzarse una excepción de tipo `IllegalArgumentException` con un mensaje descriptivo que explique el motivo del rechazo.

- **código de clase:** no puede ser nulo ni vacío.
- **duración en meses:** debe encontrarse en el rango entre 1 y 12.
- **cupo máximo:** debe ser un valor mayor que cero.

5. Operaciones del sistema

- Registrar una clase en la colección e informar por consola que fue incorporada correctamente.
- Buscar y retornar las clases cuyo código coincida con el criterio de búsqueda recibido.

6. Requerimientos de la clase principal

- Instanciar una clase de cada tipo disponible utilizando los datos de la tabla siguiente.
- Marcar la Clase Individual '**CLA-P01**' con sala privada asignada una vez creada.
- Registrar todas las clases en el gestor del sistema.
- Invocar las operaciones del gestor y mostrar sus resultados: buscar por código y listar todas las clases registradas mediante el método toString() de cada objeto.

Datos para la clase principal

Tipo	Código	Duración (meses)	Cupo	Atributo específico
Clase Individual	CLA-P01	6	1	Instrumento: Piano Evaluación de nivel: No
Clase Individual	CLA-G01	4	1	Instrumento: Guitarra Evaluación de nivel: Sí
Clase Grupal	CLA-C01	3	12	Integrantes: 8
Clase Grupal	CLA-C02	2	10	Integrantes: 5

7. Resultado esperado

La ejecución del programa con los datos indicados debe producir una salida que muestre, para cada clase, su información completa (incluyendo el costo calculado), el resultado de la búsqueda por código y el listado completo mediante toString(). Se aceptan pequeñas diferencias en los textos o en el formato de los números, siempre que se muestren todos los datos solicitados y que los resultados numéricos sean correctos.

Guía de práctica generada como material de estudio. No corresponde a una evaluación oficial.